

WEEK 1 REPORT

Requirement Analysis and Customer Database Module

Module	Customer Database
Database System	Oracle SQL
Main Operations	CREATE, INSERT, UPDATE, SELECT
Report Focus	Requirement analysis, process flow, entities and SQL implementation

This report documents the first week of work on a customer database module. It explains the requirements, identifies the entities and attributes, and records the SQL commands used to create, insert, update and retrieve customer information.

1. Requirement Analysis

Requirement analysis is the process of understanding what a system must do, what data it must store, and how users will interact with it. For the customer database module, the objective is to maintain accurate customer details in a structured and easily accessible database.

Functional Requirements

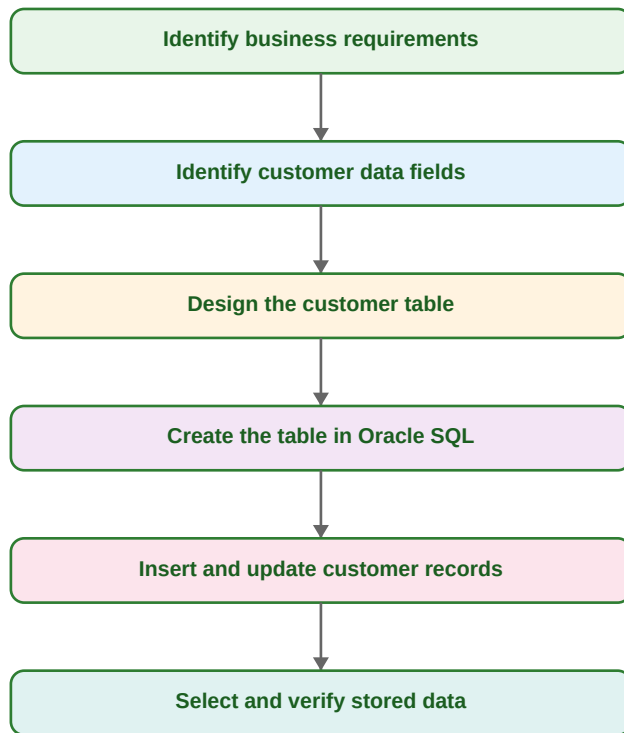
No.	Requirement
1	Create a customer table with suitable columns and data types.
2	Store a unique customer ID for every customer.
3	Record first name, last name, email, phone number and creation date.
4	Insert new customer records into the table.
5	Update existing customer information using the customer ID.
6	Retrieve and verify all stored customer records using SELECT.

Non-Functional Requirements

The database should maintain data integrity, avoid duplicate customer IDs, use appropriate data types, allow efficient retrieval, and provide reliable storage of customer information. The customer ID is defined as the primary key, ensuring that each customer can be uniquely identified.

2. Process Flow

The following flow chart shows the sequence followed while developing and testing the customer database module.



Customer database module process flow

3. Points of Definition

Term	Definition
Requirement Analysis	The activity of identifying and documenting the needs and functions of a system.
Entity	A real-world object about which information is stored, such as a customer.
Attribute	A property or characteristic of an entity, such as email or phone.
Relation/Table	A structured collection of rows and columns used to store data in a database.
Primary Key	A column or set of columns that uniquely identifies each row in a table.
VARCHAR2	An Oracle SQL data type used to store variable-length character data.
NUMBER	An Oracle SQL data type used to store numeric values.
DATE	An Oracle SQL data type used to store date and time information.
INSERT	SQL command used to add new records to a table.
UPDATE	SQL command used to modify existing records.
SELECT	SQL command used to retrieve data from one or more tables.

4. Entities

The main entity in this module is Customer. Each customer has a unique identifier and several attributes that describe the customer and the date on which the record was created.

Entity	Attribute	Description	Data Type	Constraint
Customer	customer_id	Unique identifier for a customer	VARCHAR2(20)	PRIMARY KEY
Customer	first_name	Customer's first name	VARCHAR2(20)	Optional
Customer	last_name	Customer's last name	VARCHAR2(20)	Optional
Customer	email	Customer's email address	VARCHAR2(30)	Optional
Customer	phone	Customer's phone number	NUMBER(10)	Optional
Customer	created_at	Date on which the record was created	DATE	Optional

The customer_id attribute acts as the primary key and uniquely identifies each customer record.

5. Customer Table Creation

```
SQL> create table customer(customer_id varchar2(20) primary key,first_name varchar2(20),last_name varchar2(20),email varchar2(30),phone number(10),created_at date);

Table created.

SQL> desc customer
Name                               Null?    Type
-----
CUSTOMER_ID                        NOT NULL VARCHAR2(20)
FIRST_NAME                         VARCHAR2(20)
LAST_NAME                         VARCHAR2(20)
EMAIL                             VARCHAR2(30)
PHONE                             NUMBER(10)
CREATED_AT                        DATE
```

6. Insert Values

The INSERT command adds a new customer record. In the first example, values are entered directly. In the second example, substitution variables are used so that SQL*Plus prompts the user for each value.

```
SQL> INSERT INTO customer VALUES('C-1001','Ricky','Ponting','ricky200@edss.com',9876543210,TO_DATE('2023-02-14','YYYY-MM-DD'));

1 row created.
```

```
SQL> INSERT INTO customer(customer_id,first_name,last_name,email,phone,created_at) VALUES('&cust_id','&first_name','&last_name','&email','&phone_num,DATE'&created_date');
Enter value for cust_id: C-1002
Enter value for first_name: Calvin
Enter value for last_name: Grey
Enter value for email: calvingrey@edss.com
Enter value for phone_num: 8765432109
Enter value for created_date: 2024-09-23
old 1: INSERT INTO customer(customer_id,first_name,last_name,email,phone,created_at) VALUES('&cust_id','&first_name','&last_name','&email','&phone_num,DATE'&created_date')
new 1: INSERT INTO customer(customer_id,first_name,last_name,email,phone,created_at) VALUES('C-1002','Calvin','Grey','calvingrey@edss.com',8765432109,DATE'2024-09-23')

1 row created.
```

7. Update Operation

The UPDATE command modifies an existing record. The WHERE clause identifies the customer whose phone number must be changed. Without a WHERE clause, all rows could be updated, so a precise condition is important.

```
SQL> update customer set phone = 9870654312 where customer_id = 'C-1004';

1 row updated.
```

8. Select Table

The SELECT * command retrieves all columns and rows from the customer table. The output confirms that the customer records were stored and that the update operation is reflected in the displayed data.

```
SQL> select * from customer;

CUSTOMER_ID      FIRST_NAME      LAST_NAME
-----
EMAIL            PHONE    CREATED_A
-----
C-1001           Ricky      Ponting
ricky200@edss.com 9876543210 14-FEB-23
C-1002           Calvin     Grey
calvingrey@edss.com 8765432109 23-SEP-24
C-1003           sarath     Chandar
sarathchad@edss.com 9087654321 09-NOV-23
CUSTOMER_ID      FIRST_NAME      LAST_NAME
-----
EMAIL            PHONE    CREATED_A
-----
C-1004           Amar       Mahtani
amar23@edss.com 8901234567 09-SEP-25
```

9. SQL Command Summary

Operation	Purpose	Example
CREATE TABLE	Creates the customer table and defines its columns.	CREATE TABLE customer (...);
INSERT	Adds a new customer record.	INSERT INTO customer VALUES (...);
UPDATE	Changes existing customer information.	UPDATE customer SET phone = ... WHERE customer_id = ...;
SELECT	Displays stored customer information.	SELECT * FROM customer;

Conclusion

During Week 1, the requirements of the customer database module were identified and translated into a table design. The Customer table was created successfully, records were inserted, an existing record was updated, and the stored data was retrieved using SELECT. These activities establish the basic CRUD operations required for a customer database system.